
(How) Do Language Models Track State?

Belinda Z. Li¹ Zifan Carl Guo¹ Jacob Andreas¹

Abstract

Transformer language models (LMs) exhibit behaviors—from storytelling to code generation—that seem to require tracking the unobserved state of an evolving world. How do they do this? We study state tracking in LMs trained or fine-tuned to compose permutations (i.e., to compute the order of a set of objects after a sequence of swaps). Despite the simple algebraic structure of this problem, many other tasks (e.g., simulation of finite automata and evaluation of boolean expressions) can be reduced to permutation composition, making it a natural model for state tracking in general. We show that LMs consistently learn one of two state tracking mechanisms for this task. The first closely resembles the “associative scan” construction used in recent theoretical work by Liu et al. (2023) and Merrill et al. (2024). The second uses an easy-to-compute feature (permutation parity) to partially prune the space of outputs, and then refines this with an associative scan. LMs that learn the former algorithm tend to generalize better and converge faster, and we show how to steer LMs toward one or the other with intermediate training tasks that encourage or suppress the heuristics. Our results demonstrate that transformer LMs, whether pre-trained or fine-tuned, can learn to implement efficient and interpretable state-tracking mechanisms, and the emergence of these mechanisms can be predicted and controlled. Code and data are available at <https://github.com/belindal/state-tracking>.

1. Introduction

Language models (LMs) are trained to model the surface form of text. A growing body of work suggests that model internals contain a latent, decodable state of the world—e.g., situations described by language and results of program

execution—to support prediction (Li et al., 2021; Nanda et al., 2023; Li et al., 2023). However, the mechanisms that LMs use to construct these representations are not understood. Do LMs simulate state evolution step by step across successive hidden layers or token representations (Yang et al., 2024)? Are states approximated through a complex collection of heuristics (jylin04 et al., 2024)? Is state tracking an illusion (Bender & Koller, 2020)?

This paper studies the implementation and emergence of state tracking mechanisms in language models using permutation composition as a model system: given a fixed set of objects, we train or fine-tune LMs to predict the final position of each object after a sequence of rearrangements. Previous work has used versions of this task to evaluate LMs’ empirical state tracking abilities (Li et al., 2021; Kim & Schuster, 2023; Li et al., 2023). Additionally, as shown by Barrington (1989), many complex, natural, state-tracking tasks—including simulation of finite automata and evaluation of Boolean expressions—can be reduced to permutation tracking with five or more objects. This makes it a natural model for studying state tracking in general.

Our analysis proceeds in several steps. §2 provides technical preliminaries: §2.1 and §2.2 introduce state tracking problems and the permutation composition task we use to model them (Figure 1A), and §2.3 reviews the set of interpretability tools we use to analyze LM computations. Next, §3 lays out a family of algorithms that past work has suggested LMs might, in principle, use to solve the state tracking task (Figure 1D), and describes the **signatures**—expected readouts from different interpretability methods—that we would expect to find if a given algorithm is implemented (Figure 1B-C).

Finally, §4 and §5 present experimental findings. Across a range of sizes, architectures, and pretraining schemes, we find that LMs consistently learn one of two state tracking mechanisms. The first mechanism, which we call the “associative algorithm” (AA), resembles the associative scan construction used by Liu et al. (2023) and Merrill & Sabharwal (2024) to establish theoretical lower bounds on the expressive capacity of Transformers. The second mechanism, which we call the “parity-associative algorithm” (PAA), first rules out a subset of final states using an easy-to-compute permutation parity heuristic, then uses an associative scan

¹MIT EECS and CSAIL. Correspondence to: Belinda Z. Li <bzl@mit.edu>.

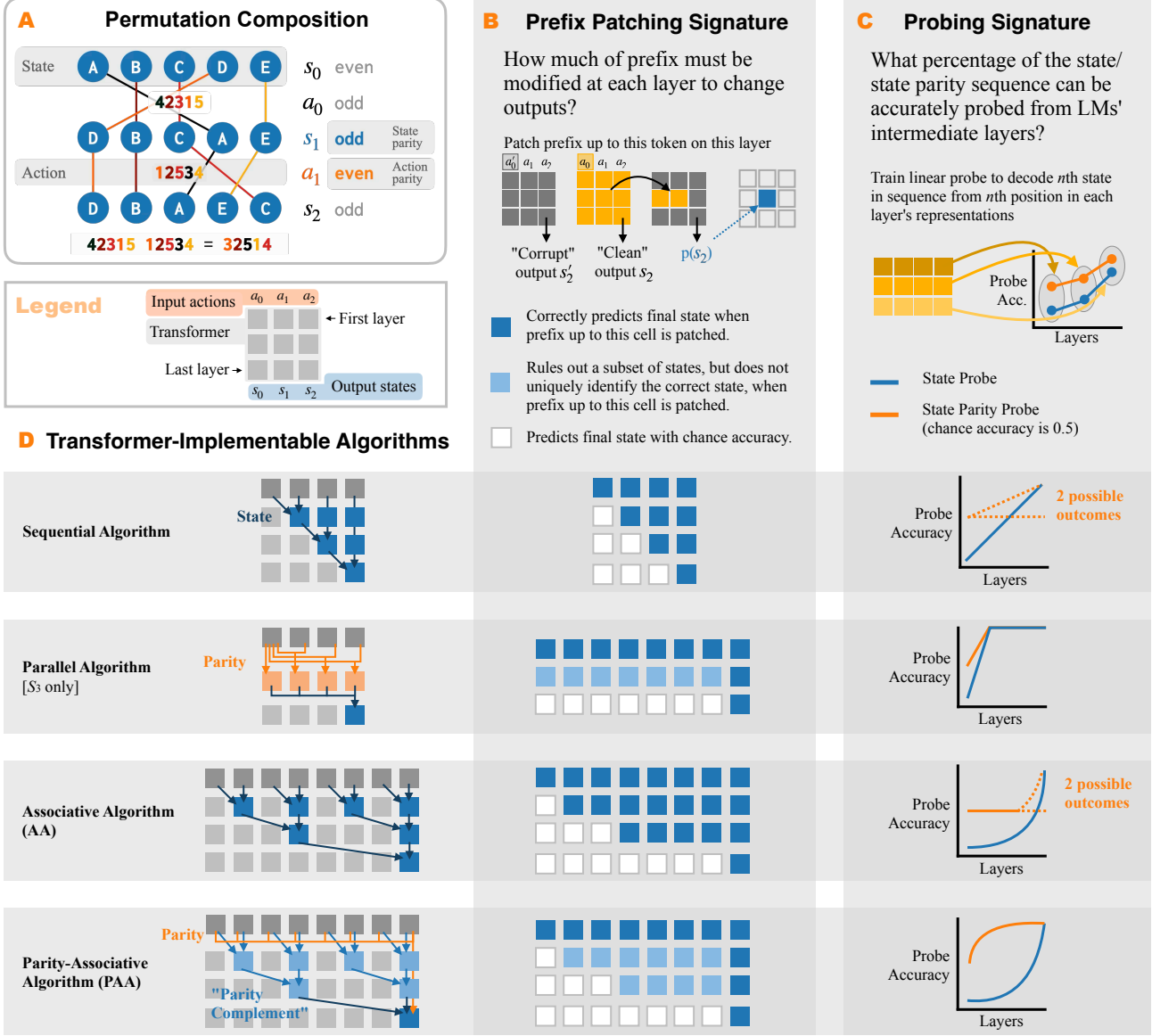


Figure 1. We use permutation word problems as a simple model of state tracking. Actions are permutations, and states are the products of those permutations; the current state can be tracked by taking the cumulative product from left to right (§2). We identify several possible algorithms that Transformers may use to solve permutation word problems: sequential, parallel, associative, and parity-associative (§3). Above, we depict the “signatures” of each algorithm under two types of interpretability analysis: *prefix patching*, where we create pairs of prompts differing only on the first token, then substitute all activation except the prefix up to a token at a particular layer, and *probing*, where we train a linear probe to map from last-token representations across the layers to either the final state or the final state parity (§2.3). Note: the dotted lines indicate two different probing signatures consistent with this algorithm (see Section C.3 for more details).

to obtain a final prediction. Notably, we fail to find evidence for either step-by-step simulation or fully parallel composition, despite their being theoretically implementable by LMs. We support our findings with evidence from representation interventions (Meng et al., 2022; Zhang & Nanda, 2024; §4.2), probes (Shi et al., 2016; §4.3), patterns in prediction errors (Zhong et al., 2024; §4.4), attention maps (Clark et al., 2019; §4.5), and training dynamics (McCoy et al., 2020;

Olsson et al., 2022; Hu et al., 2023; §5.1).

The scan operation for PAA appears difficult for LMs to implement robustly, and the choice of mechanism sometimes significantly impacts model performance on long sequences (§5.1). Whether a given LM learns AA or PAA is highly stochastic (§5.2). However, each is associated with a characteristic set of phase transitions in the training loss (Chen et al., 2024), and LMs can be steered toward one solution or

the other by training on an intermediate task that encourages or discourages LMs from learning a parity heuristic (§5.3).

As pretrained LMs sometimes re-use circuits when fine-tuned on related tasks (Prakash et al., 2024; Merullo et al., 2024), our results suggest a possible mechanism by which real-world LMs might perform state tracking when modeling language, code, and games. We show preliminary evidence of these algorithms on a version of our permutation composition tasks expressed in natural language (Section E). Looking beyond state tracking, these findings underscore both the complexity and variability of LM solutions to complex tasks, which may involve *both* heuristic features and structured solutions.

2. Background and Preliminaries

2.1. State Tracking

Inferring common ground in discourse (Li et al., 2021), navigating the environment (Vafa et al., 2024), reasoning about code (Merrill et al., 2024), and playing games (Li et al., 2023; Karvonen, 2024) all require being able to track the evolving state of a real or abstract world. There has been significant interest in understanding whether (and how) LMs can perform these tasks. In theoretical work, researchers have observed that many natural state-tracking problems (including the ones listed above) are associated with the complexity class NC^1 , but Transformers cannot track the state of arbitrarily long inputs (Merrill et al., 2022; Huet et al., 2025; Bhattamishra et al., 2020; Delétang et al., 2023; Strobl et al., 2024). However, prior work has shown that Transformers with $O(\log n)$ depth can model inputs of up to length n (Liu et al., 2023; Merrill & Sabharwal, 2024). Empirical work, meanwhile, has found that large LMs learn to solve state tracking problems (Kim & Schuster, 2023) and encode state information in their representations (Li et al., 2021; Li et al., 2023). But a mechanistic understanding of *how* trained LMs infer these states has remained elusive.

2.2. Permutation Group Word Problems

Toward this understanding, the experiments in this paper focus on one specific state tracking problem, *permutation composition*. At a high level, this problem presents LMs with a set of objects and a sequence of reshuffling operations; LMs must then compute the final order of the objects after all reshufflings have been applied (Figure 1A). Though less familiar than discourse tracking or program evaluation, Kim & Schuster (2023) used a version of this task to evaluate LM state tracking. More importantly, as shown by Barrington (1989) and recently highlighted by Merrill et al. (2024), permutation tracking (with five or more objects) is NC^1 -complete, meaning *any* other state tracking task in this family can be converted into a permutation tracking class.

This, combined with its simple structure, makes it a natural model system for studying state tracking in general.

More formally, the **finite symmetric group** S_n comprises the set of **permutations** of n objects equipped with a composition operation. For example, 42315 denotes the permutation of 5 objects (i.e. in S_5) that moves the first object to the fourth position, the second object to the second position, etc. Importantly for our findings in this paper, every permutation can be expressed as a composition of two-element **swaps** (in Figure 1A, a_0 , but not a_1 , is an example of a swap). The **parity** of a permutation (even or odd) is the parity of the number of swaps needed to create it.

The **composition** of two permutations, standardly denoted $a_1 \circ a_0$, is the result of applying a_1 after a_0 . Inputs to sequence models in machine learning are typically written with earlier inputs before later inputs (i.e., left-to-right), so for consistency with this convention, we will write $a_0 a_1$ to denote the application of a_0 *then* a_1 . Figure 1A shows the result of composing 42315 and 12534 in sequence.

Finally, the **word problem on S_n** is the problem of computing the product of a sequence of permutations. This product itself corresponds to a single permutation (32514 in Figure 1). But, following the intuition given at the beginning of the section, it may equivalently be interpreted as the final ordering of the objects being rearranged (DBAEC in Figure 1A). Following this intuitive explanation (and by analogy to other state tracking problems), we will use a_t to denote a single permutation (“action”) in a sequence, and $s_t = a_0 \cdots a_t$ to denote the result of a sequence of permutations (a “state”).

Given a sequence of permutations, we use $\epsilon(a_t)$ to denote the parity of the t th permutation, so:

$$\epsilon(s_t) = \epsilon(a_0 \cdots a_t) = \sum_i \epsilon(a_i) \mod 2 \quad (1)$$

(where ϵ is 0 for even permutations and 1 for odd ones).

All experiments in this paper train transformer language models to solve the word problem: they take as input a sequence of actions $[a_0, \dots, a_t]$, and output a sequence of state predictions $[s_0, \dots, s_t]$. We also validate our findings on a natural language version of this task in Section E, where permutations are expressed as instructions like *swap positions 2 and 3*.

2.3. Interpretability Methods

Our experiments employ several interpretability techniques to understand how LMs solve permutation word problems, which we briefly describe below. Throughout this paper, we use $h_{t,l}$ to denote the internal LM representation at token position t after Transformer layer l , with T and L denoting the maximum input length and number of layers respectively.